

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 2
з дисципліни «Професійний Python»
на тему: «Структури даних та функціональна декомпозиція»

Виконав:
студент 3 курсу
групи Фел-33
Франков Дмитро
Олександрович
Перевірив:
професор Свелеба Сергій
Андрійович

Львів-2026

Мета роботи

Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю. Опанувати роботу з list, tuple, set, dict, модулем collections, comprehensions, генераторними виразами, *args, **kwargs, lambda, замиканнями (closures) та декораторами.

Номер індивідуального варіанта

Варіант 15. Постановка задачі

Створити модуль обробки структурованих даних для аналізу логів програмної системи. Для сутності лог-запису зберігати:

- timestamp (дата і час події);
- level (рівень логування: INFO, WARNING, ERROR тощо);
- module (назва підсистеми/модуля);
- message (текст повідомлення);
- request_id (ідентифікатор запиту).

Реалізувати:

- отримання множини унікальних рівнів логування;
- підрахунок кількості подій та розподіл за рівнями через Counter;

- групування записів за модулями;
- вибірку лише подій рівня ERROR;
- швидкий пошук запису за request_id через словниковий індекс;
- рейтинг модулів за кількістю помилок;
- формування підсумкового аналітичного звіту (summary report);
- функціональну декомпозицію (щонайменше 6 функцій), замикання (closure), декоратор вимірювання часу, використання *args, **kwargs, lambda, comprehensions та експериментальний аналіз складності (benchmark).

Структура проєкту

None

Lab_02/

```
├─ .gitignore
├─ pyproject.toml
├─ README.md
├─ benchmark.py
├─ main.py
└─ src/
    └─ data_processor/
        ├── __init__.py
        ├── data.py
        ├── decorators.py
        ├── processors.py
        └─ analytics.py
```

Опис створених модулів

- **src/data_processor/__init__.py:** ініціалізує пакет обробки даних та експортує ключові функції для зовнішнього інтерфейсу.
- **src/data_processor/data.py:** містить незмінні метадані сервісу у форматі tuple (APP_METADATA) та структурований набір вхідних логів у вигляді списку словників (list[dict]).
- **src/data_processor/decorators.py:** містить декоратор @measure_time для замірювання часу виконання аналітичних функцій із використанням

time.perf_counter та functools.wraps.

- **src/data_processor/processors.py:** модуль трансформації та групування даних, що реалізує операції отримання множини рівнів (set comprehension), швидкого індексу (dict comprehension), фільтрації помилок (list comprehension), а також роботу зі структурами collections.Counter та collections.defaultdict.
- **src/data_processor/analytics.py:** модуль аналітики, який містить обчислення відсотка помилок із генераторним виразом, сортування модулів через lambda, замикання (closure) для динамічної фільтрації рівнів, агрегацію лічильників через *args, створення записів через **kwargs та генерацію підсумкового звіту.
- **benchmark.py:** модуль емпіричного тестування складності, що порівнює швидкість лінійного пошуку в list $O(n)$ та пошуку в dict $O(1)$ на масивах у 1 000, 10 000 та 100 000 записів.
- **main.py:** консольна точка входу, яка демонструє виконання всіх операцій індивідуального завдання та виводить структуровані звіти.

Текст основних фрагментів коду

src/data_processor/decorators.py

Python

```
import functools
import time

def measure_time(func):
    """Декоратор для заміру тривалості виконання операцій аналітики."""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        duration = time.perf_counter() - start_time
        print(f"[Таймінг] Функцію '{func.__name__}' виконано за {duration:.8f} c")
        return result
    return wrapper
```

src/data_processor/processors.py

Python

```
from collections import Counter, defaultdict

def get_unique_log_levels(logs: list[dict]) -> set[str]:
    """Повертає множину унікальних рівнів логуювання (Set comprehension)."""
    return {log["level"] for log in logs}

def build_request_index(logs: list[dict]) -> dict[str, dict]:
    """Створює словниковий індекс за request_id для пошуку за O(1) (Dict comprehension)."""
    return {log["request_id"]: log for log in logs}

def group_logs_by_module(logs: list[dict]) -> dict[str, list[dict]]:
    """Групує записи за модулем за допомогою defaultdict."""
    grouped = defaultdict(list)
    for log in logs:
        grouped[log["module"]].append(log)
    return dict(grouped)

def count_events_by_level(logs: list[dict]) -> Counter:
    """Підраховує кількість подій для кожного рівня логуювання (Counter)."""
    return Counter(log["level"] for log in logs)

def filter_error_events(logs: list[dict]) -> list[dict]:
    """Фільтрує записи, залишаючи виключно події рівня ERROR (List comprehension)."""
    return [log for log in logs if log["level"] == "ERROR"]
```

src/data_processor/analytics.py

Python

```
from collections import Counter
from collections.abc import Callable
from .decorators import measure_time

@measure_time
def calculate_error_rate(logs: list[dict]) -> float:
```

```

    """Обчислює частку збійних подій за допомогою генераторного виразу."""
    if not logs:
        return 0.0
    error_count = sum(1 for log in logs if log["level"] in {"ERROR", "CRITICAL"})
    return (error_count / len(logs)) * 100.0

def rank_modules_by_errors(logs: list[dict]) -> list[tuple[str, int]]:
    """Формує рейтинг модулів за кількістю помилок (використання lambda)."""
    error_logs = [log for log in logs if log["level"] == "ERROR"]
    error_counts = Counter(log["module"] for log in error_logs)
    return sorted(error_counts.items(), key=lambda item: item[1], reverse=True)

def create_level_filter(target_level: str) -> Callable[[dict], bool]:
    """Closure (замикання): повертає предикат із зафіксованим target_level."""
    normalized_target = target_level.upper()

    def predicate(log: dict) -> bool:
        return log.get("level", "").upper() == normalized_target

    return predicate

def aggregate_event_counts(*counters: Counter) -> Counter:
    """Демонстрація *args: об'єднує довільну кількість екземплярів Counter."""
    total = Counter()
    for c in counters:
        total.update(c)
    return total

def create_log_entry(**fields) -> dict:
    """Демонстрація **kwargs: створює новий словник запису логу."""

```

```

        return dict(fields)

def generate_summary_report(logs: list[dict]) -> dict:
    """Формує підсумковий звіт за всіма логами."""
    return {
        "total_events": len(logs),
        "unique_levels": {log["level"] for log in logs},
        "level_breakdown": dict(Counter(log["level"] for log in
logs)),
        "critical_count": sum(1 for log in logs if log["level"]
== "CRITICAL"),
        "error_ranking": rank_modules_by_errors(logs),
    }

```

Пояснення pyproject.toml

Файл стандарту PEP 518/621, що описує конфігурацію пакета:

- **[build-system]:** декларує використання setuptools як рушія побудови дистрибутивів.
- **[project]:** містить метадані пакета (name = "lab-data-processor", version = "0.2.0"), ліцензію та мінімальні вимоги до інтерпретатора (requires-python = ">=3.10").
- **[project.scripts]:** реєструє консольну команду log-analyzer = "main:main".
- **[tool.setuptools.packages.find]:** вказує на сканування каталогу src/ для модулів у режимі editable installation.

Вміст .gitignore

Файл виключає з індексу Git системне сміття, кеш байт-коду та каталог віртуального середовища:

```

None
.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/
build/

```

*.egg-info/

Результати виконання програми
Консольне виведення main.py

None

```
===== СИСТЕМА: OrderService (Production, v2.4.1)
=====
1. Унікальні рівні логування (set): {'ERROR', 'INFO', 'CRITICAL',
'WARNING', 'DEBUG'}
2. Кількість подій за рівнями (Counter): {'INFO': 2, 'ERROR': 3,
'WARNING': 1, 'DEBUG': 1, 'CRITICAL': 1}
3. Групування за модулями:
   - auth      : 2 подій
   - database  : 2 подій
   - payment   : 3 подій
   - kernel    : 1 подій

4. Події рівня ERROR (3 шт.):
   [2026-10-07T10:16:05] database: Connection timeout to replica
pool (req-102)
   [2026-10-07T10:17:12] payment: Card verification signature
mismatch (req-104)
   [2026-10-07T10:18:25] payment: Insufficient funds on merchant
account (req-106)

5. Результат пошуку за req-104 у dict-індексі: {'timestamp':
'2026-10-07T10:17:12', 'level': 'ERROR', 'module': 'payment',
'message': 'Card verification signature mismatch', 'request_id':
'req-104'}

6. Рейтинг модулів за кількістю помилок: [('payment', 2),
('database', 1)]

7. Фільтрація через Closure (рівень WARNING): 1 запис(ів)
[Таймінг] Функцію 'calculate_error_rate' виконано за 0.00000740 с
8. Частка збіжних подій (ERROR + CRITICAL): 50.00%
```

```
9. Агрегація двох Counter (*args): {'INFO': 2, 'ERROR': 3,
'WARNING': 1, 'DEBUG': 1, 'CRITICAL': 1}
```

```
    Створення запису через **kwargs: audit -> Compliance check
passed
```

```
===== ПІДСУМКОВИЙ ЗБИТ (SUMMARY REPORT) =====
total_events      : 8
unique_levels     : {'ERROR', 'INFO', 'CRITICAL', 'WARNING',
'DEBUG'}
level_breakdown   : {'INFO': 2, 'ERROR': 3, 'WARNING': 1,
'DEBUG': 1, 'CRITICAL': 1}
critical_count    : 1
error_ranking     : [('payment', 2), ('database', 1)]
```

Аналіз часової складності та результати benchmark

Теоретична оцінка складності:

1. **Пошук у list (лінійний):** $O(n)$ — у найгіршому випадку алгоритм перевіряє всі елементи списку послідовно.
2. **Пошук у dict за ключем:** $O(1)$ у середньому — завдяки хешуванню ключа обчислюється прямий індекс у внутрішній хеш-таблиці.
3. **Сортування рейтингу (sorted()):** $O(m \log m)$, де m — кількість унікальних модулів із помилками (алгоритм Timsort).

Результати бенчмарку (benchmark.py):

Кількість записів (n)	Лінійний пошук у list (с)	Пошук у dict-індексі (с)	Прискорення
1 000	0.00003020 с	0.00000070 с	~43x
10 000	0.00026180 с	0.00000110 с	~238x

100 000	0.00399420 c	0.00000300 c	~1331x

Висновки з бенчмарку: час лінійного пошуку в списку зростає прямо пропорційно до кількості елементів n , тоді як час доступу до словника практично не залежить від масштабу даних ($O(1)$). Попередня індексація за $O(n)$ повністю виправдовує себе при частих запитах.

Відповіді на контрольні запитання

1. Що таке list?

Впорядкована динамічна змінювана (mutable) колекція елементів довільних типів.

2. Чим list відрізняється від tuple?

list є змінюваним і динамічним; tuple — незмінюваний (immutable), має фіксований розмір і є менш вимогливим до пам'яті.

3. Чому tuple називається immutable?

Після створення кортежу неможливо змінити його довжину, додати, видалити чи перепризначити його елементи.

4. Для чого використовується set?

Для збереження множини унікальних елементів та швидкого виконання операцій об'єднання, перетину, різниці чи перевірки входження за $O(1)$.

5. Яка властивість елементів set є важливою?

Елементи обов'язково мають бути хешованими (hashable) та незмінними.

6. Що таке dict?

Змінювана асоціативна структура даних, яка зберігає пари ключ–значення.

7. Що таке key-value structure?

Організація даних, де кожному унікальному ключу зіставлене певне значення.

8. Чому пошук key у dict зазвичай швидкий?

Він базується на хеш-таблиці: хеш-функція обчислює позицію в пам'яті без необхідності перебору елементів (середня складність $O(1)$).

9. Що таке hashability?

Властивість об'єкта мати сталий хеш-код упродовж усього життєвого циклу (метод `__hash__`) і можливість коректного порівняння через `__eq__`.

10. Чому list не може бути key у dict?

Оскільки `list` є `mutable` (змінюваним), його стан і потенційний хеш можуть змінюватися, тому він не є `hashable`.

11. Для чого використовується Counter?

Спеціалізований підклас `dict` для зручного підрахунку кількості появ елементів у послідовності.

12. Для чого використовується defaultdict?

Підклас `dict`, який викликає фабричну функцію за замовчуванням при спробі доступу до ще не створеного ключа.

13. Для чого використовується deque?

Двобічна черга, оптимізована для швидкого додавання та видалення елементів з обох кінців за $O(1)$.

14. Що таке list comprehension?

Компактний синтаксис для створення нових списків шляхом ітерації та фільтрації наявних послідовностей.

15. Що таке set comprehension?

Компактний вираз для генерації унікальної множини `{expr for x in iterable}`.

16. Що таке dict comprehension?

Декларативний синтаксис для створення словника у вигляді `{key_expr: val_expr for x in iterable}`.

17. Що таке generator expression?

Вираз у круглих дужках, що ліниво генерує значення одне за одним без виділення пам'яті під усю колекцію.

18. Що означає $O(1)$?

Константна складність: час виконання операції не залежить від обсягу вхідних даних.

19. Що означає $O(n)$?

Лінійна складність: час виконання прямо пропорційний кількості елементів n .

20. Що означає $O(n \log n)$?

Лінійно-логарифмічна складність, характерна для ефективних алгоритмів

сортування (наприклад, Timsort або Merge Sort).

21. Що означає $O(n^2)$?

Квадратична складність, типова для вкладених циклів по n елементах.

22. Яка приблизна складність пошуку у list?

$O(n)$ для лінійного пошуку елемента.

23. Яка середня складність пошуку у set?

$O(1)$.

24. Яка середня складність пошуку key у dict?

$O(1)$.

25. Чому правильна структура даних може прискорити програму?

Вона мінімізує обчислювальну складність базових операцій (наприклад, перехід від $O(n)$ до $O(1)$).

26. Що таке функціональна декомпозиція?

Розподіл складної задачі на сукупність невеликих, незалежних функцій, кожна з яких має єдину зону відповідальності.

27. Які переваги малих functions?

Висока читабельність, простота тестування, відсутність побічних ефектів і можливість повторного використання.

28. Що таке positional argument?

Аргумент, значення якого зіставляється з параметром функції за порядковим номером (позицією).

29. Що таке keyword argument?

Аргумент, переданий у функцію із зазначенням імені параметра (name=value).

30. Для чого використовується *args?

Для передачі довільної змінної кількості позиційних аргументів у функцію (упаковуються в tuple).

31. Для чого використовується **kwargs?

Для передачі довільної змінної кількості іменованих аргументів у функцію (упаковуються в dict).

32. Що таке lambda function?

Коротка анонімна функція, що складається з одного виразу і повертає його

результат.

33. Де доцільно використовувати lambda?

У функціях вищого порядку як предикат чи ключ сортування (sorted(..., key=lambda ...), map, filter).

34. Чому складну business logic не варто записувати в lambda?

Це погіршує читабельність, ускладнює налагодження й унеможливорює повноцінне документування чи типізацію.

35. Що таке closure (замикання)?

Внутрішня функція, яка зберігає доступ до змінних зовнішньої функції навіть після завершення роботи останньої.

36. Які variables може пам'ятати closure?

Змінні з області видимості зовнішньої функції (non-local variables).

37. Що таке decorator?

Функція, що приймає іншу функцію як аргумент і розширює або модифікує її поведінку без безпосередньої зміни її коду.

38. Для чого застосовуються decorators?

Для логування, вимірювання часу виконання, валідації вхідних даних, кешування та контролю доступу.

39. Для чого використовується functools.wraps?

Для збереження оригінальних атрибутів та метаданих декорованої функції (назви __name__, документації __doc__ тощо).

40. Чому потрібно оцінювати складність алгоритму?

Щоб прогнозувати масштабованість, споживання ресурсів та час відгуку системи зі зростанням обсягу даних.

Висновок

У ході виконання лабораторної роботи № 2 було сформовано практичні навички роботи з базовими та спеціалізованими структурами даних Python. На основі аналізу логів програмної системи було спроектовано модульну систему обробки даних із застосуванням принципів функціональної декомпозиції. Було успішно використано list, tuple, set, dict, а також класи Counter та defaultdict модуля collections. Реалізовано замикання (closure), декоратор заміру часу та анонімні функції lambda. За допомогою експериментального бенчмарку підтверджено теоретичну складність операцій: доступ до словника за індексом $O(1)$ показав прискорення у понад 1300 разів на вибірці у 100 000 елементів порівняно з лінійним пошуком у списку $O(n)$.